

Week 1

1.

```
1 class Solution {
2     void reverseArray(int arr[]) {
3
4         int left = 0;
5         int right = arr.length - 1;
6
7         while (left < right) {
8             int temp = arr[left];
9             arr[left] = arr[right];
10            arr[right] = temp;
11
12            left++;
13            right--;
14        }
15    }
16 }
17
```

1.

```
1
2     import java.util.ArrayList;
3
4 class Solution {
5     ArrayList<Integer> getMinMax(int[] arr) {
6
7         int min = arr[0];
8         int max = arr[0];
9
10        for (int i = 1; i < arr.length; i++) {
11            if (arr[i] < min) {
12                min = arr[i];
13            }
14            if (arr[i] > max) {
15                max = arr[i];
16            }
17        }
18
19        ArrayList<Integer> result = new ArrayList<>();
20        result.add(min);
21        result.add(max);
22
23        return result;
24    }
25 }
26
```

1.

```
1
2     import java.util.ArrayList;
3
4 class Solution {
5     ArrayList<Integer> getMinMax(int[] arr) {
6
7         int min = arr[0];
8         int max = arr[0];
9
10        for (int i = 1; i < arr.length; i++) {
11            if (arr[i] < min) {
12                min = arr[i];
13            }
14            if (arr[i] > max) {
15                max = arr[i];
16            }
17        }
18
19        ArrayList<Integer> result = new ArrayList<>();
20        result.add(min);
21        result.add(max);
22
23        return result;
24    }
25 }
26
```

1.

```
1 import java.util.HashSet;
2 import java.util.ArrayList;
3 class Solution {
4     public static ArrayList<Integer> findUnion(int[] a, int[] b) {
5         HashSet<Integer> set = new HashSet<>();
6         for (int x : a) {
7             set.add(x);
8         }
9         for (int x : b) {
10            set.add(x);
11        }
12        return new ArrayList<>(set);
13    }
14 }
15
```

1.

```
1
2 class Solution {
3     public static int largest(int[] arr) {
4         int max = arr[0];
5         for (int i = 1; i < arr.length; i++) {
6             if (arr[i] > max) {
7                 max = arr[i];
8             }
9         }
10        return max;
11    }
12 }
13
14
```

2.

```
1 class Solution {
2     void rotate(int[] arr) {
3         int n = arr.length;
4         int last = arr[n - 1];
5         for (int i = n - 1; i > 0; i--) {
6             arr[i] = arr[i - 1];
7         }
8         arr[0] = last;
9     }
10 }
11
```

2.

```
1 class Solution {
2     long maxSubarraySum(int[] arr) {
3         long maxSoFar = arr[0];
4         long currentSum = arr[0];
5         for (int i = 1; i < arr.length; i++) {
6             currentSum = Math.max(arr[i], currentSum + arr[i]);
7             maxSoFar = Math.max(maxSoFar, currentSum);
8         }
9         return maxSoFar;
10    }
11 }
```

2.

```
1 class Solution {
2     public int searchInsert(int[] nums, int target) {
3         int low = 0;
4         int high = nums.length - 1;
5         while (low <= high) {
6             int mid = low + (high - low) / 2;
7             if (nums[mid] == target) {
8                 return mid;
9             }
10            else if (nums[mid] < target) {
11                low = mid + 1;
12            }
13            else {
14                high = mid - 1;
15            }
16        }
17        return low;
18    }
19 }
```

2.

```
1 import java.util.HashMap;
2 class Solution {
3     public int[] twoSum(int[] nums, int target) {
4         HashMap<Integer, Integer> map = new HashMap<>();
5         for (int i = 0; i < nums.length; i++) {
6             int complement = target - nums[i];
7             if (map.containsKey(complement)) {
8                 return new int[]{ map.get(complement), i };
9             }
10            map.put(nums[i], i);
11        }
12        return new int[]{};
13    }
14 }
```

2.

```
1 class Solution {
2     public int minJumps(int[] arr) {
3         int n = arr.length;
4         if (n == 1)
5             return 0;
6         if (arr[0] == 0)
7             return -1;
8         int maxReach = arr[0];
9         int steps = arr[0];
10        int jumps = 1;
11        for (int i = 1; i < n; i++) {
12            if (i == n - 1)
13                return jumps;
14            maxReach = Math.max(maxReach, i + arr[i]);
15            steps--;
16            if (steps == 0) {
17                jumps++;
18                if (i >= maxReach)
19                    return -1;
20                steps = maxReach - i;
21            }
22        }
23        return -1;
24    }
25 }
```